CS571: Programming Languages
Midterm 1 Practice Questions Solution

Syntax

Problem 1 Regular Expressions.

a) Define a regular expression for all strings of odd length, over the alphabet of {0,1}.

Solution: $[01]([01][01])^*$

b) Define a regular expression for identifiers over the alphabet of $\{\texttt{A}, \texttt{B}, \texttt{C}, \texttt{a}, \texttt{b}, \texttt{c}, \texttt{0}, \texttt{1}, \texttt{2}, \texttt{3}, \texttt{4}, \texttt{5}, \texttt{6}, \texttt{7}, \texttt{8}, \texttt{9}\}$, such that an identifier must begin with an alphabetic character and must contain at least one numeric character. You may use . to represent any one character from the alphabet above.

Solution:$[\texttt{ABCabc}].^*[\texttt{0}-\texttt{9}].^*$

Problem 2 Context free grammar.
Consider the following context free grammar:

$$P ::= id \mid P \vee P \mid P \wedge P \mid \neg P \mid (P)$$
$$id ::= p \mid q \mid r \mid s$$

where the set of terminals symbols is $\{p, q, r, s, \vee, \wedge, \neg, (,)\}$ and the start symbol is P.

a) Give the leftmost derivations for the following expression:

- $\neg(p \wedge q)$

Solution:

$\neg(p \wedge q) : P \Rightarrow \neg P \Rightarrow \neg(P) \Rightarrow \neg(P \wedge P) \Rightarrow \neg(id \wedge P) \Rightarrow \neg(p \wedge P) \Rightarrow \neg(p \wedge id) \Rightarrow \neg(p \wedge q)$

b) Show that this language is ambiguous.

Solution: Easy to construct two parse trees for $p \wedge q \wedge r$.

Problem 3 Context Free Grammar.
For each of the following grammars state whether it is ambiguous or unambiguous. If it is ambiguous give an equivalent unambiguous grammar. If it is unambiguous, state all the precedences and associativities enforced by the grammar. Assume Id generates identifiers.

a) $E ::= E + F \mid E - F \mid F$
$F ::= -F \mid Id \mid (E)$

Solution: unambiguous. $+$ and minus $(-)$ are left-associative, negation $(-)$ is right-associative. Negation has a higher precedence than $+$ and $-$.

b) $E ::= E \vee F \mid F$
$F ::= G \cap F \mid G \cup F \mid G$
$G ::= G \wedge H \mid H$
$H ::= Id \mid (E)$

Solution: unambiguous. $\vee, \wedge$ are left-associative, $\cap, \cup$ are right-associative.
Precedence: $\vee < \cap = \cup < \wedge$.

Names, Scopes and Bindings

Problem 4 Object allocation and lifetime.

- What is the difference between static, stack, and heap allocation and how do they affect the lifetime of a variable? See HW2 Problem 1.
- What is the meaning of the `static` keyword in C?

Problem 5 What outputs are produced by the following pseudo code if the language uses static scoping? What are the outputs if the language uses dynamic scoping?

```
a=2; b=3;
int f1(a) {
  return a + b;
}
int f2(b) {
  return 2 * f1(b);
}
print f1(a) * f2(a);
```

Solution:
Static scoping: 50 Dynamic scoping: 40

Problem 6 Consider the following pseudo-code with higher-order function support. Assume that the language has one scope for each function, and it allows nested functions (hence, nested scopes). In the pseudo code, declarations with type `int` are local declarations; otherwise the variables not declared locally are referring to the outer level of function.

```
function A() {
  int x = 5;
  function C(P) {
    int x = 3;
    P();
  }
  function D() {
    print x;
  }
  function B() {
    int x = 4;
    C(D);
  }
  B();
}
A()
```

a) Draw the symbol table for each of the five scopes (`Global`, `A`, `B`, `C`, `D`) in this program. Assume each table contains two columns: name and kind (e.g., var, fun, para).

Solution: The symbol tables are as follows:

Global	
Name	Kind
A	fun

A	
Name	Kind
x	para
B	fun
C	fun
D	fun

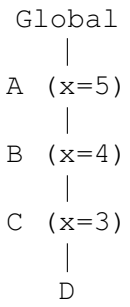
B	
Name	Kind
x	var

C	
Name	Kind
P	para
x	var

D (empty)	
Name	Kind

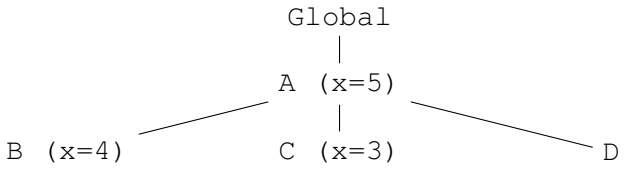
b) What would the program print if this language uses dynamic scoping and shallow binding (binding happens when the function is called)? Justify your answer by drawing the tree of symbol tables when execution reaches the print statement.

Solution: The program will print out the x in function C, which is 3. The symbol table tree is dynamic and is as follows when executing the print statement:



c) What would the program print if this language uses static/lexical scoping? Justify your answer by drawing the tree of symbol tables.

Solution: In static scoping the variable x in function D will always refer to the local variable x in function A that D is nested in. The program will print 5. The symbol table tree is static and its structure does not change over time.

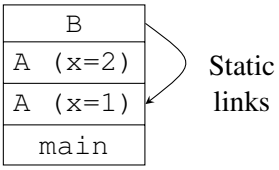


Problem 7 Consider the following pseudo-code with higher-order function support. Assume that the language has one scope for each function, and it allows nested functions (hence, nested scopes).

```
function A(x, P) {
  function B() {
    print(x)
  }
  if (x>1){
    P()
  } else{
    A(2,B)
  }
  function C() {
    int x = 4;
    print(x);
  }
}
A(1,C);
```

a) What does this program print if the language uses *static scoping* with *deep binding*? You must draw the run time stack with static links to show how you come to the conclusion.

Solution: The program will print 1.



b) What does this program print if the language uses *dynamic scoping* with *shallow binding*? You must draw the run time stack with dynamic links to show how you come to the conclusion.

Solution: The program will print 2.